Homework 2 Quantization

A GPU is recommended for quantization-aware training. You can use Google Colab for this.

Description

In this homework, you will quantize a pretrained VGG13 model on CIFAR-10. You will implement linear quantization, study how weight bit width affects accuracy, and compare post-training quantization (PTQ) with quantization-aware training (QAT). You will also extend the implementation to quantize gradients during training.

Instructions

Start with Lab_quantization.ipynb and the provided vgg_cifar.py and cifar10_vgg13_acc_94.730.pt files. In Colab, upload the model definition and checkpoint to /content/. Use PyTorch, torchvision, NumPy, and Matplotlib. Load the checkpoint and evaluate the original floating-point model as your baseline.

Requirements

Part I Linear weight quantization

Implement linear_quantization(w, bit) and quantize_model(model, bit). Follow the normalization, rounding, and dequantization steps in the notebook. Apply quantization to the weights of all convolutional and fully connected layers, and use check_model_quantization() to inspect their quantization levels.

Evaluate weight bit widths of 32, 16, 8, 6, 4, and 3, while keeping activations in floating point. Reload the original checkpoint before each experiment. Treat 32 bits as the unquantized baseline. Report the test accuracies and plot weight bit width on the x-axis and test accuracy on the y-axis. Briefly explain the trend.

Part II Weight and activation quantization

The following four tasks correspond to Tasks 1–4 in Part II of the notebook.

Task 1 Understand the provided code

Read Quantize, QuantizeW, QuantizeA, QConv, and QLinear. Briefly explain their roles, how scaling and rounding work, and why the straight-through estimator (STE) is used.

Task 2 Integrate quantization into inference

Replace nn.Conv2d and nn.Linear with QConv and Qlinear (in vgg_cifar.py), preserving the pretrained parameters and layer settings. Evaluate without retraining using nbit_w=4 and nbit_a=8. Record the PTQ test accuracy.

Task 3 Perform quantization-aware training

Starting from the same pretrained checkpoint, fine-tune the quantization-enabled model with the same weight and activation settings as Task 2.

Report the QAT test accuracy and compare it with the floating-point baseline and the PTQ result from Task 2. Record your training settings, including epochs, batch size, optimizer, learning rate, and scheduler if used. Briefly discuss whether QAT recovers accuracy and why.

Task 4 Implement gradient quantization

Extend Quantize.backward() to quantize the upstream gradients using nearest rounding. Specify the number of bits used for gradient quantization. Fine-tune and compare with the QAT result (with full-precision gradient) from Task 3, keeping the other experimental settings the same.

Optional: implement stochastic rounding for gradient quantization and compare its accuracy with nearest rounding.

Experiment notes

Use consistent test preprocessing and the full CIFAR-10 test set for final evaluation. State the random seed and experimental settings so your results can be reproduced.

The provided implementation simulates quantization using floating-point tensors and operations. Fewer quantization levels alone do not demonstrate smaller stored models or faster inference. Actual integer deployment and speed benchmarking are not required.

Submission

Submit one ZIP file containing your completed notebook, the modified vgg_cifar.py, any additional required source files, and a short PDF report. Include comments and instructions needed to reproduce your results.

Your report should include:

1. The floating-point baseline accuracy, the Part I accuracy table and curve, and representative quantization-level checks.

2. Your Task 1 explanations and a comparison table for the baseline, PTQ, QAT, and QAT with gradient quantization. Clearly label weight, activation, and gradient settings.

3. Training settings, a brief discussion of the results, and optional stochastic-rounding results if attempted.

Do not include datasets or model checkpoints in the ZIP file.

Name your ZIP file <Last-Name>_<First-Name>_HW2.zip, for example Potter_Harry_HW2.zip.

Submission Due: 23:59 EST, Thursday, 11/05/2026